

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ СИСТЕМ
УПРАВЛЕНИЯ И РАДИОЭЛЕКТРОНИКИ (ТУСУР)
Кафедра экономической математики, информатики и статистики (ЭМИС)

АБСТРАКТНЫЕ КЛАССЫ И ИНТРЕЙСЫ

Отчёт по лабораторной работе по дисциплине «Объектно-ориентированное
программирование»

Студент гр.594-2

А. А. А. А. Танцеров

“ 19 ” Октября 2025 г.

Руководитель канд. техн. наук, доцент
каф. ЭМИС ТУСУР

Я. В. Я. В. Костелей

“ ” 2025 г.

Томск 2025

Лабораторная работа №5

«Абстрактные классы и интерфейсы»

Цель работы: изучение принципов объектно-ориентированного программирования на языке C++ посредством проектирования и реализации иерархии классов, включающей абстрактные интерфейсы, базовые и производные классы.

Абстрактные классы и интерфейсы представляют собой фундаментальные механизмы объектно-ориентированного программирования для создания гибких и расширяемых архитектур. Абстрактный класс — это класс, который содержит хотя бы один чисто виртуальный метод (объявленный с синтаксисом `= 0`). Такой класс нельзя инстанцировать напрямую, его основное назначение — служить базовым классом для других, предоставляя общий интерфейс и, возможно, частичную реализацию. Чисто виртуальные методы не имеют определения в абстрактном классе и должны быть обязательно переопределены в производных классах. Это гарантирует, что все конкретные классы-наследники реализуют определённую функциональность, обеспечивая контракт между базовым и производными классами.

В контексте C++ интерфейс — это абстрактный класс, все методы которого являются чисто виртуальными (за исключением, возможно, виртуального деструктора). Интерфейсы определяют только поведение, но не его реализацию, что позволяет достичь полного разделения интерфейса и реализации. Класс может реализовывать несколько интерфейсов, что обеспечивает альтернативу множественному наследованию от полноценных классов и помогает избежать проблем вроде "алмаза смерти". Интерфейсы позволяют создавать системы, где взаимодействие между компонентами происходит через чётко определённые контракты, что повышает модульность, тестируемость и поддерживаемость кода.

Использование абстрактных классов и интерфейсов является ключевым для реализации полиморфизма в C++. Благодаря виртуальным методам, при работе с объектами через указатели или ссылки на базовый класс или интерфейс, вызывается реализация, соответствующая фактическому типу объекта, даже если он известен только во время выполнения программы. Это позволяет создавать обобщённый код, работающий с объектами различных классов через единый интерфейс, что особенно важно при проектировании плагинов, обработчиков событий или любых систем, требующих динамического расширения функциональности.

Задание. Обеспечьте возможность работы с массивом типа базового класса (массив указателей), в котором будут содержаться объекты производного и базового классов. Создайте интерфейс для вывода на консоль и оператора «[]» для установки приема. Иерархия наследования «интерфейс → базовый класс → производный класс».

На первом этапе была создана иерархия наследования "интерфейс → базовый класс → производный класс". Определён абстрактный класс IMedicine, содержащий чисто виртуальные методы для работы с информацией о приёме лекарств, включая методы доступа к данным, установки плана и факта приёма, проверки корректности. В интерфейсе также объявлен перечислимый тип TOD для времени суток и перегружен оператор [] для интуитивной работы с отметками приёма. Класс Medicine теперь наследуется от IMedicine и реализует все виртуальные методы, при этом его поля были переведены в режим protected для обеспечения доступа из производного класса.

На втором этапе реализована работа с гетерогенным массивом типа IMedicine**, содержащим указатели на объекты как базового (Medicine), так и производного (Medicines) классов. Создан демонстрационный массив, в который помещены объекты обоих типов. Для удобства работы реализованы: функция вывода информации всех объектов массива на консоль через вызов виртуального метода print(), а также использование перегруженного оператора

[] для установки отметок о приёме в конкретное время суток для любого элемента массива. Полиморфное поведение обеспечивается через виртуальные методы интерфейса IMedicine. В конце программы выполнено корректное освобождение динамической памяти для всех объектов массива и самого массива указателей.

Интерфейс класса представлен на листинге 1.

```
#pragma once
class IMedicine
{
public:
    enum TOD {
        morning,
        before_dinner,
        dinner,
        supper,
        night
    };
    virtual ~IMedicine() {};
    virtual const char* getName() const = 0;
    virtual int getDay() const = 0;
    virtual void print() const = 0;
    virtual bool& operator[](TOD time) = 0;
    virtual void setPlan(TOD time, bool shouldTake) = 0;
    virtual void setDid(TOD time, bool taken) = 0;
    virtual bool needToTake(TOD time) const = 0;
    virtual bool needToTake() = 0;
    virtual bool isCorrect(TOD time) const = 0;
    virtual bool checkPlan(TOD time) const = 0;
};
```

Листинг 1 – Интерфейс класса Medicine

Вывод: изучены принципы объектно-ориентированного программирования на языке C++ посредством проектирования и реализации иерархии классов, включающей абстрактные интерфейсы, базовые и производные классы.